Deploy GramSahayak

From two zip files to a working app on a phone: local run, MongoDB Atlas, Render or AWS hosting, and a signed APK.

The architecture skeleton and the implemented Android client are merged into one repository. The backend now serves every screen the app calls, and 51 automated endpoint checks pass against it.

Folder	What it holds	State
`android/RuralAssistant/`	Kotlin + Compose app: 8 home tiles, 11 screens, Room, WorkManager sync, trilingual UI	Implemented
`backend/`	Express API: 10 endpoints, retrieval answers, scam rules, SMS demo, voice, sync	Implemented, tested
`docs/`	Architecture, API reference, this deployment guide, Android notes	Current
`render.yaml`, `backend/Dockerfile`	One-command hosting on Render, or a container for App Runner / EC2	Ready

Read this first

Deploy in the order below. Each step ends with a check you can run. If a check fails, the fault is in that step, not the next one — fix it there before continuing.

Step	What you do	Time	Check it worked
1	Run the API locally and the smoke test	5 min	51 passed, 0 failed
2	Create MongoDB Atlas, seed the demo content	10 min	health says storeMode mongo
3	Deploy the API to Render or AWS	15 min	https URL answers /api/health
4	Point the Android app at that URL	5 min	Settings shows Connected
5	Optional: AWS speech for the voice screen	10 min	health lists aws-transcribe
6	Build and sign the APK	10 min	app-release.apk installs and runs

1. What the merge actually changed

The first zip held the architecture: a working Android shell, a backend with only `/api/health` implemented, and every other route returning HTTP 501. The second zip held the implemented Android client, which calls ten endpoints that did not exist. Merging the folders alone would have produced an app that shows an error on every screen.

So the merge did two things: the implemented client replaced the placeholder app inside the same repository layout, and the backend was written to match the client's contract exactly — same paths, same header, same response shapes.

Endpoint	Used by	Behaviour now
POST /api/query	Ask, Health	Keyword retrieval over the knowledge base, answer in the chosen language, trusted sources, grounded flag
GET /api/knowledge	offline mirror	12 trilingual entries, filter by category and free text
GET /api/schemes	Schemes	10 schemes, trilingual name, description, eligibility, how to apply
POST /api/schemes/check	Schemes	Indicative first pass: likely / unlikely / needs profile, plus a document checklist
GET /api/scholarships	Scholarships	6 scholarships, open filter, search
GET/POST/DELETE /api/scholarships/saved	Scholarships	Save and unsave per device, no duplicates
GET/POST/PUT/DELETE /api/reminders	Reminders	CRUD, idempotent on clientId so offline retries never duplicate
POST /api/scam-check	Scam checker	10 explainable rules, risk score, reasons and advice in en/hi/kn
/api/sms/preview, send, GET	SMS Centre	Simulation only: masked number, recorded, never delivered
/api/voice-sessions (+ /turns)	Virtual call	Amazon Transcribe and Polly when configured, text answer otherwise
POST /api/sync	SyncWorker	Replays the Room outbox, returns accepted and rejected operations

Three things the app needed that the skeleton did not have: a shared device identifier (`utils/DeviceId.kt`), a real `SyncWorker` that drains the Room outbox to `/api/sync`, and readable rendering for the eligibility and scam results, which previously printed raw JSON.

2. Verify the merge on your machine

Do this before touching any cloud account. It takes five minutes and removes half the possible failure modes.

```
cd gramsahayak/backend
cp .env.example .env # leave MONGODB_URI empty for now
npm install
npm run smoke # boots the API in-process and calls every endpoint
```

The last line must read **51 passed, 0 failed**. The smoke test needs no database, no AWS account and no open port: it uses the in-memory store that ships with the backend.

```
npm run dev
# [server] listening on port 5000 (development)
# [server] store: memory · speech: off

curl http://localhost:5000/api/health
curl -X POST http://localhost:5000/api/query \
-H 'Content-Type: application/json' \
-d '{"question":"fever for two days","language":"en"}'
```

> **Why the in-memory store exists**
> With MONGODB_URI empty the API seeds itself from `src/data/seed-data.js` and keeps data in memory. That makes the first demo work on any laptop, and it is why the smoke test can run in CI without secrets. Data resets when the process restarts.

3. Create the MongoDB Atlas database

Atlas is the only stateful dependency. Ten minutes, free tier, no credit card.

1. Sign in at cloud.mongodb.com and create a free **M0** cluster in Mumbai (`ap-south-1`) — closest region to your users.
2. Database Access > Add New Database User: user `gramsahayak`, autogenerate the password and copy it, role *Read and write to any database*.
3. Network Access > Add IP Address. For a hackathon demo use `0.0.0.0/0`; before public traffic, restrict it to your host's egress IPs.
4. Cluster > Connect > Drivers and copy the SRV string, adding the database name `gramsahayak` before the query string.
5. Paste it into `backend/.env` as MONGODB_URI.

```
MONGODB_URI=mongodb+srv://gramsahayak:PASSWORD@cluster0.ab1cd.mongodb.net/gramsahayak
# add ?retryWrites=true&w=majority at the end
```

If the password contains `@`, `:`, `/` or `#`, percent-encode it (`@` becomes `%40`). This single detail causes most "cannot connect" reports.

```
npm run seed # upsert 10 schemes, 6 scholarships, 12 knowledge entries
npm run seed -- --force # wipe and reload after editing seed content
npm run dev
curl http://localhost:5000/api/health
# "database": "connected", "storeMode": "mongo"
```

The server also seeds itself on first boot when the collections are empty, so a cloud deploy populates without a manual step.

4. Host the API

Three options, all covered below. Render is the fastest and includes HTTPS on the free tier; App Runner is the AWS-managed option; EC2 suits a team that already runs an instance.

Option A — Render (recommended for demo day)

1. Push the repository to GitHub. `.gitignore` already excludes `.env`, node_modules, build output, keystores and APKs.
2. Render > New > Blueprint > pick the repository. It reads `render.yaml` and creates the `gramsahayak-api` web service with `rootDir: backend`.
3. Fill the prompted secrets: MONGODB_URI, and the AWS values only if you want voice.

4 Deploy. Build takes about three minutes; the health check path is /api/health.

5 Copy the service URL, for example https://gramsahayak-api.onrender.com.

> **Free-tier cold start**
> A free Render service sleeps after 15 minutes idle, and the first request then waits about 30 seconds. Open /api/health in a browser two minutes before you present. The Starter plan removes this.

Option B — AWS App Runner from a container image

```
cd backend
export AWS_REGION=ap-south-1
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

aws ecr create-repository --repository-name gramsahayak-api --region $AWS_REGION
aws ecr get-login-password --region $AWS_REGION | docker login \
--username AWS --password-stdin $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com

docker build -t gramsahayak-api .
docker tag gramsahayak-api:latest \
$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/gramsahayak-api:latest
docker push $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/gramsahayak-api:latest
```

Then create the service with port 5000, 0.25 vCPU / 0.5 GB, health check path `/api/health`, and add MONGODB_URI plus the AWS speech variables as environment variables. App Runner issues an `https://xxxxx.ap-south-1.awsapprunner.com` endpoint.

Option C — AWS EC2 with Docker and Nginx

```
sudo apt update && sudo apt install -y docker.io
sudo systemctl enable --now docker
git clone YOUR_REPO_URL gramsahayak && cd gramsahayak/backend
sudo docker build -t gramsahayak-api .
sudo docker run -d --name gramsahayak-api --restart unless-stopped \
-p 5000:5000 --env-file /home/ubuntu/gramsahayak.env gramsahayak-api

# Nginx in front, with TLS from Let's Encrypt
sudo apt install -y nginx certbot python3-certbot-nginx
sudo certbot --nginx -d api.example.com
```

The Nginx location block needs `client_max_body_size 12m;` or voice uploads fail with HTTP 413. Proxy to `http://127.0.0.1:5000`.

Environment variables the API reads

Variable	Required	Purpose
MONGODB_URI	production	Atlas connection string; empty means the in-memory demo store
PORT	auto	Set by the host; defaults to 5000
NODE_ENV	yes	production hides internal error detail
CORS_ORIGIN	no	Comma-separated browser origins; native Android needs nothing
AWS_REGION	voice	ap-south-1
AWS_ACCESS_KEY_ID / SECRET	voice	IAM user with Polly, Transcribe, S3 permissions
AWS_S3_BUCKET	voice	Staging bucket for short clips (Transcribe needs a URI)
SPEECH_PROVIDER	no	auto (default), aws, or off

Variable	Required	Purpose
BEDROCK_MODEL_ID	no	Optional answer rewriting, e.g. amazon.nova-lite-v1:0
ALLOW_SMS_SIMULATION	no	true by default; set false to disable the SMS demo

5. Point the Android app at the live API

The app reads its backend URL at build time from `local.properties`, so the URL is compiled in — change it and rebuild, it is not a runtime setting.

```
cd android/RuralAssistant
cp local.properties.example local.properties
# edit local.properties:
# API_BASE_URL=https://gramsahayak-api.onrender.com/
```

Test target	API_BASE_URL	Extra step
Emulator, API on the same laptop	http://10.0.2.2:5000/ (default)	none
Phone over USB	http://localhost:5000/	adb reverse tcp:5000 tcp:5000, repeat after replug
Phone over Wi-Fi	http://192.168.1.20:5000/	add that IP to network_security_config.xml, open the port in your firewall
Deployed API	https://your-service/	none

Cleartext HTTP is allowed only for `10.0.2.2` and `localhost`. For a LAN address, add one line to `app/src/main/res/xml/network_security_config.xml` — otherwise the request fails silently with a cleartext error in Logcat.

```
<domain-config cleartextTrafficPermitted="true">
<domain includeSubdomains="false">10.0.2.2</domain>
<domain includeSubdomains="false">localhost</domain>
<domain includeSubdomains="false">192.168.1.20</domain> <!-- add this -->
</domain-config>
```

Open `android/RuralAssistant` in Android Studio (the folder with settings.gradle.kts, not the repository root), let Gradle sync with JDK 17 and SDK 35, then run. First launch asks for a language; Settings > Check backend should report Connected with the database state.

6. Optional: make the voice call screen live

Every other screen works without AWS. This step adds speech to text and spoken replies to the Virtual Call screen.

```
aws s3 mb s3://gramsahayak-voice-CAPITAL --region ap-south-1
# clips are deleted right after transcription and expire after a day
aws s3api put-bucket-lifecycle-configuration \
--bucket gramsahayak-voice-CAPITAL \
--lifecycle-configuration file://lifecycle.json
```

IAM policy for the backend user — grant only these actions:

```
{ "Effect": "Allow", "Action": ["polly:SynthesizeSpeech"], "Resource": "*" },
{ "Effect": "Allow",
"Action": ["transcribe:StartTranscriptionJob",
"transcribe:GetTranscriptionJob"],
"Resource": "*" },
{ "Effect": "Allow",
"Action": ["s3:PutObject", "s3:DeleteObject"],
"Resource": "arn:aws:s3:::gramsahayak-voice-CAPITAL/voice-turns/*" },
{ "Effect": "Allow", "Action": ["bedrock:InvokeModel"], "Resource": "*" }
```

Set AWS_REGION, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_S3_BUCKET and SPEECH_PROVIDER=aws on the host, redeploy, then confirm the health payload lists `aws-transcribe` and `aws-polly`.

Piece	Service	Detail
Speech to text	Amazon Transcribe	Batch job on an S3 object, mp4/AAC from MediaRecorder, polled up to 25 s
Text to speech	Amazon Polly	Voice Kajal (neural) for en-IN, hi-IN and kn-IN, MP3 returned as base64
Answer rewriting	Amazon Bedrock (optional)	Converse API rephrases retrieved facts; falls back to the stored answer
Staging	Amazon S3	Short-lived clips under voice-turns/, deleted after each turn

> **If a language is unsupported**
> A Transcribe job for an unavailable language returns FAILED. The API then replies with the explanatory text instead of audio, and the app shows it in the status line — no silent failure. Keep SPEECH_PROVIDER=off if you would rather not depend on AWS during the demo; the Ask screen still does voice input and output on the device itself.

7. Build and sign the APK

```
cd android/RuralAssistant
./gradlew assembleDebug # app/build/outputs/apk/debug/app-debug.apk
adb install -r app/build/outputs/apk/debug/app-debug.apk

keytool -genkeypair -v -keystore gramsahayak-release.jks \
-alias gramsahayak -keyalg RSA -keysize 2048 -validity 10000
# then Build > Generate Signed Bundle / APK > APK > release
```

- Back up `gramsahayak-release.jks` and its passwords. Every future update must be signed with the same key or phones refuse to install it.
- Keep the keystore out of Git — `.gitignore` already excludes `*.jks` and `*.keystore`.
- A release APK talking to a plain-HTTP LAN backend needs that IP in `network_security_config.xml` too, or it will appear to hang.
- For judges' phones, an APK built against the deployed HTTPS URL needs no other change.

8. Fifteen-minute check before you present

Run these in order. Any failure points at one specific step above.

```
curl https://YOUR-SERVICE/api/health
curl https://YOUR-SERVICE/api/schemes | head -c 200
curl -X POST https://YOUR-SERVICE/api/query \
-H 'Content-Type: application/json' \
-d '{"question":"which schemes help small farmers","language":"kn"}'
```

1. Language selection > Kannada; Home shows the connection chip as online.
2. Settings > Check backend > Connected, with the database state shown.
3. Ask > a Kannada question > answer plus trusted sources > Read answer aloud.
4. Health > an English health question > answer.
5. Schemes > search farmer > open a scheme > Check general eligibility > summary and documents.
6. Scholarships > save one > it appears in the saved list.
7. Reminders > add one with 2026-11-01T09:00:00+05:30 > complete it > delete it.
8. Scam checker > paste an OTP/KYC message > HIGH RISK with reasons and advice.
9. SMS Centre > valid number, tick consent > "Simulated — recorded only".
10. Virtual call (if AWS is on) > Start call > Record > Stop and send > transcript, answer, audio.
11. Airplane mode on, Settings > Sync now; when the network returns the queued rows replay.

9. Troubleshooting

Symptom	Cause	Fix
App: "Unable to connect to backend"	wrong base URL	Emulator needs 10.0.2.2, not localhost. Re-sync Gradle after editing local.properties
CLEARTEXT communication not permitted	HTTP host not whitelisted	Add the host to network_security_config.xml, or use HTTPS
adb reverse worked, phone fails	tunnel dropped on replug	Re-run adb reverse tcp:5000 tcp:5000
health: database disconnected	Atlas allowlist or password	Add the egress IP, re-check the URI, percent-encode special characters
503 DATABASE_UNAVAILABLE	Mongo connection dropped	The API stays up by design; restore Atlas access and retry
Schemes list empty	collections not seeded	npm run seed, or restart (auto-seeds when empty)
First request takes 30 s	Render free cold start	Warm /api/health before the demo
Voice returns text only	no AWS creds or bucket	Complete step 6, or use the Ask screen
Polly failed (403)	IAM policy incomplete	Add polly:SynthesizeSpeech and restart
413 on voice upload	proxy body limit	client_max_body_size 12m, or lower MAX_AUDIO_BYTES
Gradle sync fails	SDK 35 or JDK missing	Install SDK Platform 35, set Gradle JDK to 17

10. What is still open

State this plainly if a judge asks. The demo is honest by design; the gaps are known and named in the code.

- **Content is seeded, not official.** 10 schemes, 6 scholarships and 12 knowledge entries, all marked `verified: false`. Confirm amounts and deadlines on the official portal before quoting them.
- **Answers are extractive.** Retrieval returns stored text, and when nothing matches it says so rather than inventing. Bedrock only rephrases what retrieval found.
- **Read paths are not cached.** Room stores the tables and the outbox, and SyncWorker replays writes, but a phone with no connectivity shows only what it already loaded.
- **SMS is a simulation.** No gateway, no delivery, numbers stored masked.

- **The Android app has not been compiled here.** It was assembled without an Android SDK; Gradle sync in Android Studio is its first real build.
- **Harden before public traffic:** add helmet and rate limiting on /api/query, /api/scam-check and /api/sms/*, authenticate before any real SMS gateway, pin the Atlas allowlist, and move secrets into the platform's secret store.

> **Demo in one line**
> Ask a health question in Kannada, check a scheme, flag a scam SMS, then say: "the same API runs on Render today and on AWS App Runner from the same container, and the app works with no backend at all for stored screens."